

claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_4fd1a4ea-6c0\agent-ac27c83481434b0cf.jsonl`  
- SHA-256: `428cf95574c1d85a07c10f68a5551e8dece48cbe520ccac7c10bae797fe9a60e`  
- Source modified: `2026-07-02T09:29:46+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_4fd1a4ea-6c0`  
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`
```

Transcript

1. user (2026-07-02T09:20:08.329Z)

You are reviewing a god-module extraction for IMPORT/MODULE-LOAD mechanics.
REFACTOR UNDER REVIEW (uncommitted, in worktree C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs, branch refactor/p23-orchestration-split off origin/master):

- NEW backend/orchestration.py (739 lines): 10 members moved VERBATIM from backend/main.py: `_finalize_reingest`, `_report_config_for_input`, `_ingest_remote_segments`, `_cloud_available`, `_maybe_dispatch_remote`, `_execute_processing`, `_remote_input_size_bytes`, `_CompileError`, `_resolve_compile`, `_execute_compile`.
- backend/main.py 2360 -> 1703 lines; re-exports all 10 names via "from backend.orchestration import (...)" (noqa F401);
5 now-unused imports removed (csv, math, Tuple, StitchingConfig, localize_watermark).
- Enumerated intentional deltas (everything else is AST-identical, machine-verified):
(1) `_maybe_dispatch_remote`: "req: ProcessRequest" -> string forward-ref "ProcessRequest" (TYPE_CHECKING import).
(2) `_execute_processing`: inserted "import backend.main as `_main`" after docstring; `global_config/db_instance/ProcessRequest` resolved via `_main.<name>` at call time (the `job_worker` seam convention).
(3) two "# type: ignore[...]" comments (AST-invisible).
- NEW tests/test_main_module_budget.py: line budget 1750, route ceiling 18, re-export identity check.
- Verified already: AST-identity (10 moved + 51 retained members), ruff+format clean, mypy green (142 files),
ratchet test passes. Full suite running separately.

Inspect with: `git -C "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs" diff` (and `git -C "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs" status`); the new files are untracked/modified in the worktree.
READ-ONLY: do not modify the worktree, do not checkout/commit.

Hunt for:

1. Circular-import hazards: backend/main.py imports backend.orchestration at module level (where exactly? check placement relative to what orchestration needs); orchestration must NOT import backend.main at module level ? verify (only call-time/TYPE_CHECKING). Simulate the load order mentally for: "import backend.main", "import backend.orchestration" FIRST (e.g. a test importing orchestration directly), "python -m backend.main" (the `__main__` double-module trap at the bottom of main.py ? does the trap still hold? does orchestration's lazy "import backend.main as `_main`" inside `_execute_processing` resolve the RIGHT module object when running under `python -m`?).
2. Import-order side effects: main.py does `load_dotenv` + `logging.basicConfig` at import; orchestration does `logging.getLogger` at import. Any test importing backend.orchestration WITHOUT backend.main would previously have triggered main's side effects when importing these symbols ? could any consumer now skip side effects it relied on?
3. The removed imports (csv/math/Tuple/StitchingConfig/localize_watermark) ? confirm nothing in

retained main.py or any external consumer references them via backend.main (e.g. m.StitchingConfig).

4. wheel/packaging: pyproject entry point backend.main:main unchanged; any MANIFEST/py_modules listing that must include the new module? (check pyproject packaging config picks up backend/orchestration.py).

Report ONLY real, evidence-backed findings (file:line).

2. user (2026-07-02T09:20:16.895Z)

```
M backend/main.py
?? backend/orchestration.py
?? tests/test_main_module_budget.py
refactor/p23-orchestration-split
```

3. user (2026-07-02T09:20:24.153Z)

```
"""Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of #390).
```

The full hash -> validate -> segment -> report pipeline (``\_execute\_processing``), the Cloud Run dispatch + ingest path (``\_maybe\_dispatch\_remote`` / ``\_ingest\_remote\_segments``), and the shared compile resolution + worker handler (``\_resolve\_compile`` / ``\_execute\_compile``).

Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is unchanged.

SEAM DISCIPLINE (same convention as ``backend.api.job\_worker``): anything that must remain monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional ``db\_instance``/``global\_config`` globals and the ``ProcessRequest`` model) is resolved through the ``backend.main`` module object at call time (``import backend.main as \_main``), never as a bare local. ``backend.main`` re-exports every name below, so ``monkeypatch.setattr(backend.main, ...)`` and ``import backend.main as m; m.\_execute\_processing(...)`` keep working unchanged. This module must NOT import ``backend.main`` at module level (main imports us for the re-export).

```
"""
```

```
import csv
import json
import logging
import math
import os
import tempfile
from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
from urllib.parse import quote

from backend import storage
from backend.config import AppConfig, StitchingConfig
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.pipeline.stitcher.watermark_i18n import localize_watermark
from backend.pipeline.validators.base import registry
from backend.reporting import emit_indexer_report
from backend.results import Failure
from backend.utils.hashing import compute_video_id
from backend.utils.paths import output_base, safe_output_filename

if (
    TYPE_CHECKING
):
    # runtime import would be circular (main imports us for the re-export)
    from backend.main import ProcessRequest  # noqa: F401

logger = logging.getLogger(__name__)
```

```
def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
    """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
```

```

On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark) and
keep the prior good results intact ? a failed/empty re-run never destroys prior data.
"""
if segments:
    db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
else:
    db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)

def _report_config_for_input(config: Any, raw_input: str) -> Any:
    """Return report config that keeps remote-input reports out of disposable scratch."""
    storage_config = (
        config.get("storage")
        if isinstance(config, dict)
        else getattr(config, "storage", None)
    )
    try:
        if storage.input_is_local(raw_input, storage_config):
            return config
    except ValueError:
        return config
    try:
        cfg = (
            config.model_dump(mode="json")
            if hasattr(config, "model_dump")
            else dict(config or {})
        )
    except Exception:
        cfg = {}
    report = dict(cfg.get("report") or {})
    if report.get("output_dir"):
        return config
    report["output_dir"] = os.path.join(output_base(config), "reports")
    cfg["report"] = report
    return cfg

def _ingest_remote_segments(
    db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
) -> int:
    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the bucket
    and
    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
    produced.
    Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst pattern
    (cloud
    backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a malformed row
    is
    skipped, non-finite / overflowing values are rejected, the loop is bounded, and the temp dir
    is
    cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as a job
    failure
    and prunes any partial rows (destroy-AFTER-confirm)."""
    uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
    ref = storage.parse_uri(uri)
    n = 0
    with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
        local = storage.fetch(
            uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
        )
        with open(local, newline="", encoding="utf-8") as f:
            for row in csv.DictReader(f):
                if n >= max_rows:

```

```

        logger.warning(
            "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the rest.",
            max_rows,
            uri,
        )
    break
try:
    start, end = float(row["start"]), float(row["end"])
except (KeyError, TypeError, ValueError):
    continue
if not (math.isfinite(start) and math.isfinite(end)):
    continue # reject inf/nan times rather than persist a junk rally
meta: Dict[str, Any] = {
    "source": "cloud_run",

```

4. user (2026-07-02T09:20:36.637Z)

```

1:"""Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of #390).
6:Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is unchanged.
9:monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
11:the ``backend.main`` module object at call time (``import backend.main as _main``), never as
a
12:bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13:...))`` and ``import backend.main as m; m._execute_processing(...)`` keep working unchanged.
14:This module must NOT import ``backend.main`` at module level (main imports us for the re-
export).
40:    from backend.main import ProcessRequest # noqa: F401
355:    import backend.main as _main # call-time seam: main's globals/model stay patchable
361:        if req is None and db is not None and isinstance(db, _main.ProcessRequest):
364:        elif req is None and isinstance(config, _main.ProcessRequest):
369:        config = _main.global_config
371:        db = _main.db_instance

```

5. user (2026-07-02T09:20:44.968Z)

```

17:from dotenv import load_dotenv
20:# launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None) keeps
the
21:# exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare load_dotenv().
25:load_dotenv(env_file_to_load())
28:logging.basicConfig(
301:# dispatch+ingest, compile resolution + worker handler) now lives in backend.orchestration
306:from backend.orchestration import ( # noqa: F401 (re-exports: names stay addressable on
backend.main)
1693:if __name__ == "__main__":
1694:    # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
1696:    # module object. If `main()` runs here it configures `__main__`'s globals, leaving
1700:    # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-fire.)

```

6. user (2026-07-02T09:20:54.576Z)

```

import argparse
import json
import logging
import os
import shutil
import socket
import sys
import tempfile
import threading
import uuid
import webbrowser

```

```

from importlib.metadata import PackageNotFoundError
from importlib.metadata import version as _pkg_version
from urllib.parse import quote

```

```

import uvicorn
from dotenv import load_dotenv

```

PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None) keeps exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare load_dotenv stdlib-only import ? keeps this pre-pydantic line light.

```

from backend.utils.app_home import env_file_to_load, resolve_output_dir

```

```

load_dotenv(env_file_to_load())

```

Centralized logging setup for the indexer (hot paths use logger, CLI output uses print)

```

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%H:%M:%S",
)
logger = logging.getLogger(__name__)
from typing import Any, Dict, List, Optional

from fastapi import FastAPI, HTTPException, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
from starlette.middleware.trustedhost import TrustedHostMiddleware

from backend import (
    billing, # M8: per-job cost ledger (USD internal / INR display)
    storage, # M2: URI-aware input acquisition (local = identity passthrough)
)
from backend.api import (
    auth as edge_auth, # M9: Cloudflare Access edge-auth (default-OFF)
)
from backend.config import (
    AppConfig,
    StartupCheckError,
    enforce_startup_checks,
    write_light_default_config, # P2b: batteries-included first-run config seeding
)
from backend.config import ( # new typed config
    load_config as load_app_config,
)
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.pipeline.validators.base import registry
from backend.providers import provider_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure # standardized error/result contract
from backend.storage.capabilities import (
    storage_capabilities, # P3: secret-free "which media can this box use" report
)
from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
    ffmpeg_bin,
    ffprobe_bin,
)

```

```

from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
from backend.utils.hashing import compute_video_id # canonical file-identity hashing
from backend.utils.paths import (
    output_base,
    resolve_under_root,
    safe_output_filename,
    validate_input_path,
)

```

7. user (2026-07-02T09:21:01.918Z)

```

)
from backend.utils.redact import (
    redact_secrets, # P0b: mask secret-shaped fields in /api/config
)
from backend.utils.single_instance import acquire_lock # P0e: one instance per database

app = FastAPI(title="Sports Highlight Indexer API")

```

```

def create_app(config: AppConfig, db: Database) -> FastAPI:
    """Factory: inject shared state via ``app.state`` so route handlers access it through
    ``request.app.state`` ? eliminating the Optional module-level globals that forced
    ``backend.main`` and ``backend.api.job_worker`` into mypy quarantine.

```

```

    Also sets the module-level ``global_config`` / ``db_instance`` for transitional
    compatibility (helpers and tests still reference them). Route handlers use
    ``request.app.state`` exclusively."""
    global global_config, db_instance
    global_config = config
    db_instance = db
    app.state.config = config
    app.state.db = db
    return app

```

```

def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
    """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-separated),
    default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
    ``backend.main``
    does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to its
    origin."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
    origins = [o.strip() for o in raw.split(",") if o.strip()]
    return origins or ["*"]

```

CORS for the web UI. `allow_origins='*' with allow_credentials=True is rejected by browser fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so credentials off. The origin list is configurable via the env var above (default ["*"]) so a hosted deployment lock it to its origin; the middleware is fixed at startup.`

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=_cors_origins_from_env(),
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

```

def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:

```

```

    """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
    web page can point a domain it controls at 127.0.0.1 and script requests to the local
server;
    the browser sends ``Host: attacker.com``, which this rejects (the server only answers
loopback
    Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the default
keeps
    Starlette's TestClient host ``testserver`` so the in-process suite isn't broken. Read from
the
    ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
    deploy sets its public host (or ``*`` when fronted by an auth gate that validates the
host)."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
    hosts = [h.strip() for h in raw.split(",") if h.strip()]
    # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
    # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("[:1]:port"), so IPv6
    # loopback is NOT supported by this guard ? access the appliance via 127.0.0.1/localhost.
    return hosts or ["localhost", "127.0.0.1", "testserver"]

```

DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are accepted (a remote page can't read responses cross-origin AND its Host header is rejected here) so an unexpected ``www.`` host is rejected rather than 307-redirected.

```

app.add_middleware(
    TrustedHostMiddleware,
    allowed_hosts=_allowed_hosts_from_env(),
    www_redirect=False,
)

def _resolve_bind_host(
    config: Optional[AppConfig] = None, cli_host: Optional[str] = None
) -> str:
    """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
    when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS an auth
gate
    in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a routable
interface with no auth gate is unsafe, hence the localhost default; widen it consciously."""
    if cli_host and cli_host.strip():
        return cli_host.strip()
    env = os.environ.get("RALLY_BIND_HOST")
    if env and env.strip():
        return env.strip()
    sec = getattr(config, "security", None)
    if getattr(sec, "edge_auth_enabled", False):
        return "0.0.0.0" # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
    return "127.0.0.1"

```

--- P2: the friendly ``indexer --app`` launcher (cross-platform, no native component ? O1)

```

def _port_is_free(host: str, port: int) -> bool:
    """True if (host, port) can be bound right now (best-effort; a TOCTOU race is acceptable ?
uvicorn surfaces a real bind failure loudly)."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

```

```

    try:
        s.bind((host, port))
        return True
    except OSError:
        return False

def _find_free_port(host: str) -> int:
    """An OS-assigned free port on host."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind((host, 0))
        return int(s.getsockname()[1])

def _resolve_app_port(host: str, port: int) -> int:
    """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never blocks
    the app."""
    return port if _port_is_free(host, port) else _find_free_port(host)

def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
    """Poll `health_url` until the server is up, then open `url` in the user's default browser.
    Returns True if a browser open was attempted. No-op (returns False) when RALLY_NO_BROWSER is
    set
    (tests / headless) or the server never becomes healthy (don't open a dead page). Never
    raises."""
    if os.environ.get("RALLY_NO_BROWSER"):
        return False
    import time as _time
    import urllib.request

    deadline = _time.monotonic() + timeout
    while _time.monotonic() < deadline:
        try:
            with urllib.request.urlopen(health_url, timeout=2) as r:
                if getattr(r, "status", 200) == 200:
                    break
        except Exception:
            _time.sleep(0.3)
    else:
        return False
    try:
        webbrowser.open(url)
    except Exception:
        pass
    return True

```

Serves static frontend files directly (now under api/ per approved structure).
PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching
arbitrary directory finds the shipped assets instead of creating a stray ./backend/api/s
(The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)

```

_STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
try:
    os.makedirs(_STATIC_DIR, exist_ok=True)
except OSError:
    # The dir ships with the package (a no-op create); guard against a read-only install
    # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
    # PermissionError.
    pass
app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")

```


P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed SPA (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only if the SPA is absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so no CORS is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets is sufficient (no client-side path routing ? no history-fallback catch-all, which would also be a route).

```
_BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
_UI_DIR = (
    _BUNDLED_UI_DIR
    if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
    else _STATIC_DIR
)
_UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
if os.path.isdir(_UI_ASSETS_DIR):
    app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")

@app.get("/", response_class=HTMLResponse)
def serve_index():
    index_path = os.path.join(_UI_DIR, "index.html")
    if os.path.exists(index_path):
        with open(index_path, "r", encoding="utf-8") as f:
            return f.read()
    return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"

def _bundled_ui_version() -> Optional[Dict[str, Any]]:
    """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is bundled.
    Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
    guard)."""
    path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
    try:
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    except (OSError, ValueError):
        return None
```

Global variables initialized at startup

```
db_instance: Optional[Database] = None
global_config: Optional[AppConfig] = None # now typed (Pydantic model)
```

P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if it is not, GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI in).

```
_instance_lock: Optional[Any] = None
```

Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION now lives in backend.api.job_worker. Re-exported here so the names stay addressable in `backend.main` (`JobWaiting`, `\_drain\_due\_jobs`, `\_arm\_wake\_timer`, `\_get\_executor`, `\_run\_claimed\_job`, `\_MAX\_DRAIN\_WAKE\_SEC`, ...)). The worker resolves these seams by holding onto this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still intercepts the inter-function calls (a parked job arms the PATCHED `\_arm\_wake\_timer` so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.

```
from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable on
    backend.main)
    _MAX_DRAIN_WAKE_SEC,
```

```

    JobWaiting,
    _arm_wake_timer,
    _drain_due_jobs,
    _get_executor,
    _job_to_request,
    _maybe_record_job_cost,
    _run_claimed_job,
    _schedule_next_drain_if_waiting,
)

```

The processing/compile ORCHESTRATION (hash?validate?segment?report pipeline, dispatch+ingest, compile resolution + worker handler) now lives in backend.orchestration (P23 god-module split; re-cut of #390). Re-exported here so the names stay addressable in backend.main ? `import backend.main as m; m.\_execute\_processing(...)` , the job worker call-time `\_main.\_execute\_compile(...)` seam, and monkeypatch.setattr(backend.main, ...), all keep working unchanged.

```

from backend.orchestration import ( # noqa: F401 (re-exports: names stay addressable on
backend.main)
    _cloud_available,
    _CompileError,
    _execute_compile,
    _execute_processing,
    _finalize_reingest,
    _ingest_remote_segments,
    _maybe_dispatch_remote,
    _remote_input_size_bytes,
    _report_config_for_input,
    _resolve_compile,
)

```

Legacy thin wrapper for any remaining direct calls during transition.

New code should import load_app_config / AppConfig from backend.config directly.

```

def load_config(config_path: str = "config.json") -> Dict[str, Any]:
    cfg = load_app_config(config_path)
    return cfg.model_dump(mode="json")

```

--- API Models ---

```

class ProcessRequest(BaseModel):
    video_path: str
    player_pool: Optional[List[str]] = None

```

8. user (2026-07-02T09:21:11.970Z)

```

    # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
    _get_executor().submit(_drain_due_jobs, global_config, db_instance)
    # Stamp shared state on the app so route handlers access it via request.app.state
    # instead of the Optional module-level globals (CODE_CLEANUP_AUDIT P5 / D2).
    create_app(global_config, db_instance)
    uvicorn.run(app, host=bind_host, port=port)

if __name__ == "__main__":
    # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads` resolve
    the
    # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ? a
    SEPARATE

```

```

# module object. If `main()` runs here it configures `__main__`'s globals, leaving
# `backend.main.db_instance` = None; the async-job drain then crashes silently on
# `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch through
# `backend.main` so `main()` configures the module everyone actually reads. (No recursion:
# imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-fire.)
import backend.main as _bm

_bm.main()
328:class ProcessRequest(BaseModel):

```

9. user (2026-07-02T09:21:27.702Z)

```

backend/api/job_worker.py:3:Extracted verbatim from backend.main to shrink the orchestrator;
behavior is unchanged.
backend/api/job_worker.py:11:`backend.main` module object at call-time (`import backend.main as
_main`), never as a
backend/api/job_worker.py:12:bare local. `backend.main` re-exports these names, so
`monkeypatch.setattr(backend.main,
backend/api/job_worker.py:14:`_arm_wake_timer`, so no real daemon timer leaks in tests).
Importing `backend.main` lazily
backend/api/job_worker.py:77:     import backend.main as _main
backend/api/job_worker.py:105:    import backend.main as _main
backend/api/job_worker.py:221:    import backend.main as _main
backend/api/job_worker.py:241:    import backend.main as _main
backend/api/job_worker.py:258:    import backend.main as _main
backend/api/uploads.py:1:"""Chunked upload (B2, PR-2) ? extracted from backend.main into its own
router.
backend/api/uploads.py:10:The startup config lives on ``backend.main.global_config`` (set ONLY
in ``main()``).
backend/api/uploads.py:11:``_upload_cfg`` resolves it through the ``backend.main`` module object
so it always
backend/api/uploads.py:13:``backend.main.global_config`` flow through unchanged).
backend/api/uploads.py:50:    import backend.main as _main
backend/utils/app_home.py:19:stdlib-only (os, platform, pathlib) so it is safe to import at
``backend.main`` line 1 ? before
backend/worker/__main__.py:13:This is a thin orchestrator: it composes the SAME building blocks
the ``backend.main`` CLI uses

```

10. user (2026-07-02T09:21:43.189Z)

```

orchestration OK; backend.main loaded? False
main OK

```

11. user (2026-07-02T09:21:57.402Z)

```

./backend/eval/batch.py:10:or any API. The orchestration/side-effects live in
`scripts/run_phase3_eval.py`.
./scripts/install.py:140:# --- IO / orchestration
-----
./tests/test_main_module_budget.py:4:landed in it. The P23 extraction moved the
processing/compile orchestration to
./tests/test_main_module_budget.py:12:- **Re-export contract:** every orchestration name stays
addressable on ``backend.main``
./tests/test_main_module_budget.py:27:# main.py was 1703 lines right after the P23 orchestration
extraction (2026-07-02).
./tests/test_main_module_budget.py:85:def test_orchestration_names_stay_addressable_on_main():
./tests/test_main_module_budget.py:90:    import backend.orchestration as o
./tests/test_main_module_budget.py:94:        f"backend.main.{name} is not the
backend.orchestration.{name} object ? "
./tests/test_profile_parity.py:4:windows + the same segment ranges** regardless of the active
deployment profile / orchestration.
./tests/test_profile_parity.py:15: * **L4c** ? the worker (`cmd_infer`, the cloud-serving
orchestration) writes an `intervals.csv`
---

```

```
149:         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
316:         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
317:         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
```

12. user (2026-07-02T09:22:10.357Z)

```
---external---
./backend/orchestration.py:40:     from backend.main import ProcessRequest # noqa: F401
./tests/test_locale_analytics.py:12:from backend.main import ProcessRequest
```

13. user (2026-07-02T09:22:19.988Z)

```
tests/test_api_annotations.py
tests/test_api_assets.py
tests/test_api_cloud_dispatch.py
tests/test_api_endpoints.py
tests/test_app_home.py
tests/test_app_launcher.py
tests/test_async_jobs.py
tests/test_billing.py
tests/test_bind_and_host.py
tests/test_config.py
tests/test_edge_auth.py
tests/test_flywheel.py
tests/test_import_smoke.py
tests/test_job_worker_loop.py
tests/test_job_worker_module_seam.py
tests/test_light_tier_no_torch.py
tests/test_locale_analytics.py
tests/test_main_module_budget.py
tests/test_provider_gemini.py
tests/test_server_entrypoint.py
tests/test_startup_checks.py
tests/test_ui_bundle.py
tests/test_upload_download.py
```

14. user (2026-07-02T09:22:42.909Z)

```
1 tests/test_provider_gemini.py:264:setattr(main
1 tests/test_main_module_budget.py:14:setattr(backend.main
1 tests/test_job_worker_module_seam.py:74:setattr(m,
1 tests/test_job_worker_module_seam.py:7:setattr(m,
1 tests/test_job_worker_module_seam.py:51:setattr(m,
1 tests/test_job_worker_module_seam.py:48:setattr(m,
1 tests/test_job_worker_loop.py:50:setattr(m,
1 tests/test_job_worker_loop.py:46:setattr(m,
1 tests/test_edge_auth.py:233:setattr(m,
1 tests/test_config.py:330:setattr(main
---
```

15. user (2026-07-02T09:24:00.679Z)

```
"""THE HAZARD PIN: a drain that finds a not-yet-due 'waiting' job must arm the
wake-timer THROUGH `backend.main._arm_wake_timer`, so patching it on
`backend.main` intercepts and no live daemon timer is created. Patch
`m._arm_wake_timer` (the documented seam), run a real drain, and assert it was
the patched recorder that fired ? not a bare local that would leak a timer."""
from backend.infrastructure.database import Database
```

```
db = Database(str(tmp_path / "t.db"))
monkeypatch.setattr(m, "db_instance", db)
m.app.state.db = db
armed: list = []
monkeypatch.setattr(m, "_arm_wake_timer", lambda delay, *a: armed.append(delay))
```

```

db.create_job("j1", "/v.mp4")
db.set_job_waiting("j1", delay_sec=3600, progress="parked")

ran = m._drain_due_jobs(m.global_config, m.db_instance)

assert ran == 0 # not due -> not claimed
assert db.get_job("j1")["status"] == "waiting"
# The patched seam fired (clamped to the cap) ? proves the indirection holds.
assert armed and 0.0 < armed[-1] <= m._MAX_DRAIN_WAKE_SEC

def test_arm_wake_timer_submits_drain_through_patched_executor(monkeypatch):
    """`_arm_wake_timer` must submit the re-drain through `backend.main._get_executor`
    and `backend.main._drain_due_jobs`, so a stubbed executor + a real (non-leaking)
    timer let us observe the scheduled callback without sleeping or leaking threads."""
    submitted: list = []

    class _RecordingExecutor:
        def submit(self, fn, *a, **k):
            submitted.append((fn, a, k))

    monkeypatch.setattr(m, "_get_executor", lambda: _RecordingExecutor())

    captured = {}

    class _FakeTimer:
        def __init__(self, delay, fn):
            captured["delay"] = delay

@pytest.fixture
def env(tmp_path, monkeypatch):
    db = Database(str(tmp_path / "t.db"))
    cfg = AppConfig()
    monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
    # Stub the only real-timer seam so a parked job never arms a live daemon timer; tests
    # that care about scheduling assert against this recorder instead.
    armed: list = []
    monkeypatch.setattr(m, "_arm_wake_timer", lambda delay, *a: armed.append(delay))
    app = m.create_app(cfg, db)
    client = TestClient(app)
    return client, db, tmp_path, monkeypatch, armed

```

16. user (2026-07-02T09:24:12.560Z)

```

tests/test_api_annotations.py:91: monkeypatch.setattr(m, "_remote_input_size_bytes", lambda
*a, **k: len(payload))
tests/test_api_assets.py:129: monkeypatch.setattr(m, "_remote_input_size_bytes", lambda *a,
**k: len(payload))
tests/test_api_cloud_dispatch.py:128: """Make backend.compute.get_compute_backend (re-
imported inside _maybe_dispatch_remote) build a
tests/test_api_cloud_dispatch.py:151: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:170: m._execute_processing(
tests/test_api_cloud_dispatch.py:196: m._execute_processing(
tests/test_api_cloud_dispatch.py:209: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:228: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:269: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:298: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:312: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:353: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:396: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:427: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:445: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:457: def test_capabilities_reports_cloud_available(cloud_env):
tests/test_api_cloud_dispatch.py:602: out = m._execute_processing(

```

```

tests/test_api_cloud_dispatch.py:624: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:648: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:676: out = m._execute_processing(
tests/test_api_cloud_dispatch.py:704: out =
m._execute_processing(m.ProcessRequest(video_path=str(video)))
tests/test_api_endpoints.py:815: mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k:
len(payload))
tests/test_api_endpoints.py:833: assert m._report_config_for_input(local_cfg, str(tmp_path /
"local.mp4")) is local_cfg
tests/test_api_endpoints.py:839: assert m._report_config_for_input(explicit,
"gs://bucket/videos/x.mp4") is explicit
tests/test_api_endpoints.py:851: mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k: 10
* 1024**3)
tests/test_api_endpoints.py:868: mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k:
None)
tests/test_async_jobs.py:159: """Item 5 positive: direct CLI-style path
(m._execute_processing) vs job-worker path
tests/test_async_jobs.py:173: direct_out = m._execute_processing(m.global_config,
m.db_instance, req)
tests/test_async_jobs.py:207: direct_f = m._execute_processing(m.global_config,
m.db_instance, req_f)
tests/test_async_jobs.py:226: de = m._execute_processing(m.global_config, m.db_instance,
req_e)
tests/test_async_jobs.py:489: def test_worker_dispatches_compile_kind_to_execute_compile(env):
tests/test_async_jobs.py:490: """The claimed-job dispatcher routes kind='compile' to
_execute_compile (NOT _execute_processing),
tests/test_async_jobs.py:503: mp.setattr(m, "_execute_compile", _fake_compile)
tests/test_async_jobs.py:506: "_execute_processing",
tests/test_async_jobs.py:507: lambda *a, **k: pytest.fail("a compile job must not hit
_execute_processing"),
tests/test_base.py:135: Exercise the exact duck-typing patterns used in main.py (both the
_execute_processing
tests/test_base.py:181: This allows the except blocks in main.py (_execute_processing:193-198
and the
tests/test_job_worker_module_seam.py:6:(`_arm_wake_timer`, `_get_executor`, `db_instance`,
_execute_processing`)
backend/api/job_worker.py:9:every cross-function seam ? `db_instance`, `global_config`,
_execute_processing`,
backend/api/job_worker.py:60: is the segmenter's `_execute_processing`, run UNDER the
destroy-after-confirm contract.
backend/api/job_worker.py:88: # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
backend/api/job_worker.py:114: result = _main._execute_compile(
backend/api/job_worker.py:122: result = _main._execute_processing(
scripts/nightly_reelcount.py:430: # backend/main.py:_execute_processing) ? play_segments
has an FK to videos.id, so a

```

17. user (2026-07-02T09:24:25.236Z)

```

backend/main.py:307: _cloud_available,
backend/main.py:308: _CompileError,
backend/main.py:311: _finalize_reingest,
backend/main.py:312: _ingest_remote_segments,
backend/main.py:313: _maybe_dispatch_remote,
backend/main.py:314: _remote_input_size_bytes,
backend/main.py:315: _report_config_for_input,
backend/main.py:316: _resolve_compile,
backend/main.py:338: # default (deployment.compute_target). Only honoured for these two
values; see _maybe_dispatch_remote.
backend/main.py:473: "cloud_available": _cloud_available(cfg),
backend/main.py:643: need = _remote_input_size_bytes(input_ref, getattr(cfg, "storage",
None))
backend/main.py:872: need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage",
None))
backend/main.py:1017: need = _remote_input_size_bytes(

```

```

backend/main.py:1170:         video, _kept, out_name = _resolve_compile(
backend/main.py:1177:     except _CompileError as e:
backend/main.py:1618:         _finalize_reingest(db_instance, video_id, segments,
play_wm, cand_wm)
backend/main.py:1628:         config=_report_config_for_input(global_config,
args.video_path),
backend/orchestration.py:4:Cloud Run dispatch + ingest path (`_maybe_dispatch_remote` /
`_ingest_remote_segments`),
backend/orchestration.py:5:and the shared compile resolution + worker handler
(`_resolve_compile` / `_execute_compile`).
backend/orchestration.py:45:def _finalize_reingest(db, video_id: str, segments, play_wm: int,
cand_wm: int) -> None:
backend/orchestration.py:58:def _report_config_for_input(config: Any, raw_input: str) -> Any:
backend/orchestration.py:86:def _ingest_remote_segments(
backend/orchestration.py:136:def _cloud_available(cfg) -> bool:
backend/orchestration.py:155:def _maybe_dispatch_remote(
backend/orchestration.py:204:    # _cloud_available so we fail clearly rather than silently
running local); None / unknown ? default.
backend/orchestration.py:215:        if not _cloud_available(cfg):
backend/orchestration.py:298:        _ingest_remote_segments(db, video_id, result.outputs_uri,
storage_dict)
backend/orchestration.py:306:    # Only the NEW cloud rows (id > the pre-run watermark);
_finalize_reingest then drops the old ones.
backend/orchestration.py:310:    _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
backend/orchestration.py:411:    # _maybe_dispatch_remote when it dispatches. None ? nothing ran
(validation/config fail).
backend/orchestration.py:497:        remote = _maybe_dispatch_remote(
backend/orchestration.py:534:        _finalize_reingest(db, video_id, segments, play_wm,
cand_wm)
backend/orchestration.py:578:        config=_report_config_for_input(config,
req.video_path),
backend/orchestration.py:593:def _remote_input_size_bytes(input_ref, storage_config) ->
Optional[int]:
backend/orchestration.py:614:class _CompileError(Exception):
backend/orchestration.py:624:def _resolve_compile(
backend/orchestration.py:631:    ``_CompileError(404/400)``. Returns ``(video_row,
kept_segments, safe_out_name)``. """
backend/orchestration.py:634:        raise _CompileError(404, "Indexed video record not found.")
backend/orchestration.py:639:        raise _CompileError(400, str(e)) from e
backend/orchestration.py:644:        raise _CompileError(400, "No matching play segments found
to stitch.")
backend/orchestration.py:665:        raise _CompileError(
backend/orchestration.py:686:        video, _kept, out_name = _resolve_compile(
backend/orchestration.py:689:    except _CompileError as e:

```

18. user (2026-07-02T09:24:54.260Z)

```

tests/test_api_cloud_dispatch.py:311:     with caplog.at_level("WARNING", logger="backend.main"):
tests/test_api_cloud_dispatch.py:444:     with caplog.at_level("WARNING", logger="backend.main"):
tests/test_app_launcher.py:124:         "backend.main",
tests/test_app_launcher.py:185:     [sys.executable, "-m", "backend.main", "--app", "--port",
str(port)],
tests/test_config.py:212:     with caplog.at_level(logging.WARNING,
logger="backend.config.loader"):
tests/test_config.py:230:         with caplog.at_level(logging.WARNING,
logger="backend.config.loader"):
tests/test_config_deployment.py:188:     with caplog.at_level(logging.WARNING,
logger="backend.config.loader"):
tests/test_config_deployment.py:241:     with caplog.at_level(logging.WARNING,
logger="backend.config.loader"):
tests/test_config_deployment.py:255:     with caplog.at_level(logging.WARNING,
logger="backend.config.loader"):
tests/test_config_deployment.py:288:     with caplog.at_level(logging.INFO,
logger="backend.config.loader"):
tests/test_config_deployment.py:297:     with caplog.at_level(logging.INFO,

```

```

logger="backend.config.loader"):
tests/test_database.py:538:    with caplog.at_level(logging.ERROR,
logger="backend.infrastructure.database"):
tests/test_database.py:547:    with caplog.at_level(logging.ERROR,
logger="backend.infrastructure.database"):
tests/test_database.py:558:    with caplog.at_level(logging.ERROR,
logger="backend.infrastructure.database"):
tests/test_database.py:568:    with caplog.at_level(logging.ERROR,
logger="backend.infrastructure.database"):
tests/test_database.py:579:    with caplog.at_level(logging.ERROR,
logger="backend.infrastructure.database"):
tests/test_database.py:590:    with caplog.at_level(logging.ERROR,
logger="backend.infrastructure.database"):
tests/test_execution_policy.py:214:    with caplog.at_level(logging.DEBUG,
logger="test.execution.policy"):
tests/test_gemini.py:194:    with caplog.at_level(logging.ERROR,
logger="backend.pipeline.segmenters.gemini"):
tests/test_gemini.py:231:    with caplog.at_level(logging.ERROR,
logger="backend.pipeline.segmenters.gemini"):
---pyproject---
3:# Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4:# (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5:# reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6:# auto-detection would otherwise miss the un-__init__'d `backend` root.
14:build-backend = "hatchling.build"
37:    # Used directly on the serving path (backend/features/rally_seq.py: uniform_filterid /
80:# Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
81:# these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
84:# MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
88:[project.scripts]
89:# `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
90:indexer = "backend.main:main"
98:# `backend` is a namespace package (no __init__.py); list both import roots
99:# explicitly so an editable/wheel install exposes `import backend` and
101:packages = ["backend", "training"]

```

19. user (2026-07-02T09:25:05.947Z)

```

)
assert out["status"] == "failed"
assert "could not read the result" in out["message"]
assert db.query_play_segments(out["video_id"], {}) == []

def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
    """If the worker re-hashes different bytes (result.video_id != local id), ingest still
    proceeds
    under the LOCAL id but logs a WARNING so the orphaned-report condition is diagnosable."""
    db, _bucket, monkeypatch = cloud_env
    _inject_fake(monkeypatch, _FakeCloudClient(video_id="DIVERGED"))
    with caplog.at_level("WARNING", logger="backend.main"):
        out = m._execute_processing(
            m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
        )
    assert out["status"] == "success"
    assert len(db.query_play_segments(out["video_id"], {})) == 2
    assert any(
        "DIVERGED" in r.getMessage() and "!= local" in r.getMessage()
        for r in caplog.records
    )

class _InProcSeg:
    """Minimal in-process segmenter (writes one play row) ? proves the local path actually

```



```

ran."""

def __init__(self, db_, cfg_):
    self.db = db_

def process_video(self, vid, path, *a, **k):
    sid = self.db.add_play_segment(vid, 0.0, 5.0)
    assert "isn't configured" in out["message"]

def test_unknown_compute_value_falls_back_to_server_default(cloud_env, caplog):
    """An unrecognised compute value is ignored (logged) and the server default applies ? here a
    cloud_run server still dispatches to the cloud."""
    db, _bucket, monkeypatch = cloud_env
    fake = _FakeCloudClient()
    _inject_fake(monkeypatch, fake)
    with caplog.at_level("WARNING", logger="backend.main"):
        out = m._execute_processing(
            m.global_config,
            m.db_instance,
            m.ProcessRequest(
                video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="banana"
            ),
        )
    assert out["status"] == "success"
    assert len(fake.calls) == 1 # fell back to the configured cloud_run default
    assert any("unknown compute" in r.getMessage() for r in caplog.records)

```

20. user (2026-07-02T09:25:21.060Z)

```

[project.scripts]
`indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
indexer = "backend.main:main"

```

[tool.hatch.metadata]

The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct reference*); hatchling requires opting in to allow those in metadata.

```
allow-direct-references = true
```

[tool.hatch.build.targets.wheel]

`backend` is a namespace package (no `__init__.py`); list both import roots explicitly so an editable/wheel install exposes `import backend` and `import training` regardless of cwd.

```
packages = ["backend", "training"]
```

21. user (2026-07-02T09:25:35.894Z)

```

def _cloud_available(cfg) -> bool:
    """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item 3).

    True when the control plane is wired for Cloud Run: either the configured default target is
    already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env coordinates
    (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
    (``storage.output_root``)
    are present so a forced per-request override can actually dispatch + collect a result.

    Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it would
    work."""
    deployment = getattr(cfg, "deployment", None)
    if (getattr(deployment, "compute_target", "") or "") == "cloud_run":

```

```

        return True
    has_job = bool(
        os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
    )
    out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
    return bool(has_job and out_root)

def _maybe_dispatch_remote(
    cfg,
    db,
    req: "ProcessRequest",
    video_id: str,
    seg_name: str,
    val_results,
    play_wm: int,
    cand_wm: int,
    notify,
) -> Optional[Tuple[Dict[str, Any], bool]]:
    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
    deployment.compute_target
    == "cloud_run", run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally intervals
    into
    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.

    Returns `(response_dict, ran)` when the cloud path is taken ? `ran` is whether the cloud
    job
    actually EXECUTED (`False` for a pre-dispatch config rejection, so the observability
    report
    doesn't claim a fake segmentation run) ? or `None` to fall through to the in-process
    segmenter
    (**DEFAULT-OFF*: `get_compute_backend` returns `None` for every non-`cloud_run`
    target, so
    the in-process path stays byte-identical)."""
    from backend.compute import ComputeJobSpec, get_compute_backend

    def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
        # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows (id >
        play_wm);
        # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
        db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
        # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run" only when
        the
        # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker rc /
        ingest
        # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target` records
        the
        # intended backend, `requested` the picker choice, `dispatched` whether the remote job
        ran.

        return (
            {
                "video_id": video_id,
                "status": "failed",
                "message": msg,
                "results": [r.model_dump() for r in val_results],
                "play_segments": [],
                "compute": {
                    "path": "cloud_run" if ran else "not_run",
                    "requested": req.compute,
                    "target": "cloud_run",
                    "dispatched": ran,
                },
            },
            ran,
        )

```

```

# SPA compute-picker (item 3): a per-request choice overrides the server's configured
default.
# "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch (guarded by
# _cloud_available so we fail clearly rather than silently running local); None / unknown ?
default.
choice = (req.compute or "").strip().lower()
if choice and choice not in ("local", "cloud"):
    logger.warning(
        "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
        req.compute,
    )
    choice = ""
if choice == "local":
    return None # explicit "run on my machine" ? in-process regardless of the server's
target
if choice == "cloud":
    if not _cloud_available(cfg):
        return _failed(
            "cloud-serving was requested but this server isn't configured for it "
            "(no Cloud Run target). Pick 'run on my machine', or configure cloud serving.",
            False,
        )
    compute = get_compute_backend(cfg, target_override="cloud_run")
else:
    compute = get_compute_backend(
        cfg
    ) # server default (default-OFF unless cloud_run)
if compute is None:
    return None # default-OFF ? run the in-process segmenter below (unchanged)

storage_cfg = getattr(cfg, "storage", None)
return (
    {
        "video_id": video_id,
        "status": "success",
        "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
        "results": [r.model_dump() for r in val_results],
        "play_segments": segments,
        "compute": provenance,
    },
    True,
)

def _execute_processing(
    config=None, db=None, req=None, progress=None
) -> Dict[str, Any]:
    """The full hash ? validate ? segment ? report pipeline for one video.

    Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
    module-level ``global_config``/``db_instance`` (backward-compatible with existing
    tests that pass only ``req``). Route handlers always pass explicit params; older
    test code that calls ``_execute_processing(req)`` still works.

    ``progress`` is an optional callable(stage: str) used to surface coarse job progress.
    Raises on unexpected internal errors ? the caller records those as a job failure
    """
    import backend.main as _main # call-time seam: main's globals/model stay patchable

    # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
    # shift positional args ? req arrives in position 0 (config), progress in position 1 (db).
    if config is not None and not isinstance(config, AppConfig):
        # Old signature: _execute_processing(req, progress=None)

```

```

    if req is None and db is not None and isinstance(db, _main.ProcessRequest):
        req = db
        db = None
    elif req is None and isinstance(config, _main.ProcessRequest):
        req = config
        config = None
# Resolve config/db from module globals when not passed explicitly
if config is None:
    config = _main.global_config
if db is None:
    db = _main.db_instance

notify = progress or (lambda stage: None)

notify("hashing")
# M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged (identity ?
# byte-identical to today); a bucket/Drive URI is fetched to scratch first. The original
# req.video_path (the source URI) is kept for the DB record + report; only the file-reading
# consumers (hash / validators / segmenter) use the resolved local path.
local_video = ""
try:
    local_video = storage.acquire_local_input(
        req.video_path, getattr(config, "storage", None)
    )
    video_id = _main.compute_video_id(local_video)
except Exception:
    storage.cleanup_acquired_local_input(
        req.video_path, local_video, getattr(config, "storage", None)
    )
    raise
try:
    existing_video = db.get_video(video_id)
except Exception:
    storage.cleanup_acquired_local_input(
        req.video_path, local_video, getattr(config, "storage", None)
    )
    raise
was_reingest = existing_video is not None

# typed AppConfig: attribute access for the default segmenter (with safe fallback)
default_seg = getattr(
    getattr(config, "indexing", None), "default_segmenter", "motion"
)
seg_name = req.segmenter or default_seg
segmenter_actual = None
segmentation_ran = False
val_results: List[Any] = []
val_context: Dict[str, Any] = {}
# Compute-path PROVENANCE (the validation method): which backend actually ran the DETECT ?
# "in_process" once the local segmenter is reached, set on the response as "cloud_run" by
# _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config fail).
compute_actual: Optional[str] = None
response: Optional[Dict[str, Any]] = None
try:
    if (
        existing_video
        and existing_video.get("status") == "processed"
        and not req.force
    ):

```

22. user (2026-07-02T09:25:56.634Z)

```

362:         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
365:         elif req is None and isinstance(config, _main.ProcessRequest):
370:         config = _main.global_config

```

```
372:         db = _main.db_instance
386:         video_id = _main.compute_video_id(local_video)
571:         paths = _main.emit_indexer_report(
702:     compiler = _main.VideoCompiler(
```

23. user (2026-07-02T09:26:14.355Z)

```
16:``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
31:from backend.config import AppConfig, StitchingConfig
33:from backend.pipeline.stitcher.watermark_i18n import localize_watermark
386:         video_id = _main.compute_video_id(local_video)
571:         paths = _main.emit_indexer_report(
647:     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
679:     import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
695:     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
701:     localized_watermark = localize_watermark(stitching.watermark, locale)
702:     compiler = _main.VideoCompiler(
===
warning: in the working copy of 'backend/main.py', LF will be replaced by CRLF the next time Git
touches it
350:-         video_id = compute_video_id(local_video)
535:-         paths = emit_indexer_report(
664:-     compiler = VideoCompiler(
```

24. user (2026-07-02T09:26:33.233Z)

Object-attribute patches (``m.storage.fetch``, ``m.segmenter\_registry.get``, ...) hit the shared objects and need no routing.

```
"""
```

```
import csv
import json
import logging
import math
import os
import tempfile
from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
from urllib.parse import quote

from backend import storage
from backend.config import AppConfig, StitchingConfig
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.watermark_i18n import localize_watermark
from backend.pipeline.validators.base import registry
from backend.results import Failure
from backend.utils.paths import output_base, safe_output_filename

if (
    TYPE_CHECKING
): # runtime import would be circular (main imports us for the re-export)
    from backend.main import ProcessRequest # noqa: F401

logger = logging.getLogger(__name__)
```

25. assistant (2026-07-02T09:26:46.577Z)

The worktree changed between my reads (parent agent is iterating). Let me re-verify the current state cleanly.

26. user (2026-07-02T09:26:58.090Z)

warning: in the working copy of 'backend/main.py', LF will be replaced by CRLF the next time Git touches it

```
backend/main.py | 757 ++++-----  
1 file changed, 50 insertions(+), 707 deletions(-)  
All checks passed!  
orch-first OK, main loaded? False  
identity: True
```

27. user (2026-07-02T09:27:17.770Z)

```
python -m exit: 0  
<frozen runpy>:128: RuntimeWarning: 'backend.main' found in sys.modules after import of package  
'backend', but prior to execution of 'backend.main'; this may result in unpredictable behaviour  
dispatched through: backend.main | trap holds: True
```

28. user (2026-07-02T09:27:35.976Z)

```
61:from backend.pipeline.stitcher.compiler import VideoCompiler  
64:from backend.reporting import emit_indexer_report  
74:from backend.utils.hashing import compute_video_id # canonical file-identity hashing  
347: # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is written  
verbatim.  
349: csv: str  
359: # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content hash.  
746: ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5 ``video_id`` ?  
so it joins  
760: name = safe_output_filename(f"{req.video_id}.rallies.csv")  
770: f.write(req.csv or "")  
773: 1 for ln in (req.csv or "").splitlines() if ln[:1].isdigit() and "," in ln  
822: MD5 (`compute_video_id`, D3) so it joins the pipeline by content hash, and appears in  
891: video_id = compute_video_id(local_working)  
1437: compiler = VideoCompiler(  
1559: video_id = compute_video_id(local_video)  
1621: paths = emit_indexer_report(  
===tests===  
tests/test_api_endpoints.py:264: vid = m.compute_video_id(str(vid_file))  
tests/test_api_endpoints.py:300: vid = m.compute_video_id(str(vid_file))  
tests/test_api_endpoints.py:340: vid = m.compute_video_id(str(vid_file))  
tests/test_api_endpoints.py:659: vid = m.compute_video_id(str(vid_file))  
tests/test_api_endpoints.py:922: assert job["video_id"] == m.compute_video_id(  
tests/test_api_endpoints.py:1053: vid = m.compute_video_id(str(ref_file))  
tests/test_async_jobs.py:166: vid = m.compute_video_id(str(vid_file))  
tests/test_async_jobs.py:193: vid2 = m.compute_video_id(str(vid_file2))  
tests/test_async_jobs.py:300: vid = m.compute_video_id(str(vid_file))  
tests/test_async_jobs.py:311: vid = m.compute_video_id(str(vid_file))  
tests/test_async_jobs.py:357: vid = m.compute_video_id(str(vid_file))  
tests/test_async_jobs.py:406: vid = m.compute_video_id(str(vid_file))  
tests/test_async_jobs.py:425: vid = m.compute_video_id(str(vid_file))  
tests/test_job_worker_loop.py:104: vid = m.compute_video_id(str(vid_file))  
tests/test_rally_seg_eval.py:257: "trajectory": _write_oscillating_traj(tmp_path /  
"m.csv"),  
backend/orchestration.py:386: video_id = _main.compute_video_id(local_video)  
backend/orchestration.py:571: paths = _main.emit_indexer_report(  
backend/orchestration.py:702: compiler = _main.VideoCompiler(  

```

29. user (2026-07-02T09:28:20.492Z)

```
backend/orchestration.py:209: "[API] ignoring unknown compute=%r (expected  
'local' | 'cloud'); using server default.",  
backend/orchestration.py:286: "[API] cloud video_id %s != local %s ? input bytes  
differ between the API hash "  
warning: in the working copy of 'backend/main.py', LF will be replaced by CRLF the next time Git  
touches it  
backend/main.py | 757 ++++-----  
1 file changed, 50 insertions(+), 707 deletions(-)  
M backend/main.py
```

```
?? backend/orchestration.py
?? tests/test_main_module_budget.py
```

30. user (2026-07-02T09:28:37.088Z)

```
..... [100%]
5 passed in 2.76s
```

31. user (2026-07-02T09:29:02.776Z)

```
150:         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
317:         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
318:         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
27:from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
39:     TYPE_CHECKING
744
```

32. user (2026-07-02T09:29:11.014Z)

```
275:db_instance: Optional[Database] = None
276:global_config: Optional[AppConfig] = None # now typed (Pydantic model)
306:from backend.orchestration import ( # noqa: F401 (re-exports: names stay addressable on
backend.main)
328:class ProcessRequest(BaseModel):
```

33. user (2026-07-02T09:29:45.952Z)

Structured output provided successfully